



AuditWare

X

Swapboard

Smart Contract Security Review

Performed by **Auditware** for **ECF**



auditware.io



audit_wizard



Review Summary

The Auditware team has performed a security review on Swapboard.

Swapboard is a minimalist, trustless OTC bulletin board for peer-to-peer ERC20 token swaps on Ethereum. The protocol implements an immutable orderbook with zero fees, no admin functions, and atomic execution guarantees.

The following items were in scope for the audit:

- Smart contract vulnerabilities in `contracts/src/`
- Frontend vulnerabilities that could lead to fund loss
- Subgraph data integrity issues

The following items were explicitly defined out of scope:

- Known behaviors documented in "Trust Assumptions" and "Known Limitations" sections
- Third-party dependencies (OpenZeppelin, ethers.js) - report to upstream maintainers
- Theoretical attacks without proof of concept or economic viability analysis
- Social engineering, phishing, or UI spoofing attacks
- Issues requiring compromised private keys
- Network-level attacks (eclipse attacks, BGP hijacking)
- Vulnerabilities in the testnet deployment or test tokens (TestToken.sol)
- Frontend issues that do not lead to fund loss (cosmetic bugs, UX issues)
- Subgraph indexing delays or temporary data inconsistencies
- Gas optimization suggestions (unless they enable griefing attacks)

Audit commit hash: [9b4eed2](#)

18 Feb 2026 Retest

During February 2026 the team was asked to perform a retest to attest that the findings originally brought in this report are fixed. Results of the retest are tracked on the finding table or extended finding comments.

Please note that there remain proven exploits affecting the protocol as described above, that were out of scope for the test and retests.

Retest commit hash: [0c0a7ee](#)



Contract Overview

Protocol Trust Assumptions

The contract operates under these assumptions:

- The contract trusts that ERC20 tokens behave correctly. Malicious tokens (e.g., tokens with transfer hooks, blacklists, or admin functions) can cause unexpected behavior or fund loss.
- Users are responsible for verifying token contract addresses and implementations before trading.
- Order creation time relies on block timestamps, which miners can manipulate within ~15 seconds.
- The contract has no price feeds. Users set their own prices and are responsible for fair pricing.
- The contract cannot be upgraded or paused. Critical bugs require migration to a new contract.

Protocol Accepted Risks

It's important to note that there are high-risk design limitations not mentioned in this report, as they were classified as protocol accepted risks, such as:

- Orders can be front-run by MEV bots. Users should consider using private mempools for large orders.
- Rebasing tokens cause permanent fund lockup. If selling token rebases down after order creation, the maker cannot fill the order and cannot cancel to retrieve funds - the tokens become permanently stuck in the contract with no recovery option. Positive rebases also leave excess tokens trapped in the contract across multiple orders. Users must avoid rebasing tokens entirely.
- Malicious token implementations can result in total fund theft. Specifically, if the buying token (tokenB) returns true on transfers but doesn't actually move tokens, the taker receives the full selling token amount while the maker receives nothing.
- Fee-on-transfer tokens are blocked for selling token (tokenA) via balance checks, but allowed for buying token (tokenB) at maker's risk. If tokenB has transfer fees enabled after order creation, maker receives less than the expected amount.
- Tokens with blacklists or pausability can prevent order fulfillment or cancellation if the maker or taker gets blacklisted, or if the token gets paused. The funds may become locked until the token is unpaused or the address is removed from the blacklist.
- Orders remain active until filled or cancelled. There is no automatic expiration mechanism.
- Users pay gas for all operations. Failed transactions (e.g., insufficient allowance) still cost gas.

We recommend re-visiting those risks as soon as possible in the protocol's lifetime, and until then to **notably warn users** and ensure they understand and acknowledge these risks before wiring funds through the protocol.



Findings Overview

Finding	Component	Risk Level	Retest Status
AW-H-01: Insufficient Input Validation	Frontend	High	Fixed
AW-M-01: Forced Browsing	Frontend	Medium	Fixed
AW-M-02: Insufficient Contract Security Tests	Contract	Medium	Fixed
AW-L-01: Missing decrement logic in OrderFilled/OrderCanceled handlers inflates token order counts	Subgraph	Low	Fixed



AW-H-01: Insufficient Input Validation

Severity: **High**

Status: **Fixed**

Retest notes:

Fixed by adding forward slash escaping to `escapeHtml()` and replacing all `innerHTML` usage with safe `textContent` / `createTextNode` methods.

Locations:

- <https://ethcf.github.io/swapboard/#revoke-list> (source)
- [ETHCF/swapboard/frontend/app.js#L2796](#) (source)
- [ETHCF/swapboard/frontend/app.js#L1134](#) (sink)

Description:

During the audit, it was found that the protocol deploys a simple html/js/css setup as the protocol's frontend.

Although standard UI frameworks such as React, Vue, or others are not mandatory for secure development, and sometimes present their own risks, they do provide a high level of built-in input sanitization, and control over when input sanitization is dangerous (e.g. via `"dangerouslySetInnerHTML"`). The swapboard implementation accommodates for this by its internal html escaping mechanism [escapeHtml](#), however, two issues were found:

- Although most of the characters to escape according to the [OWASP XSS prevention cheatsheet](#) are implemented, escaping of forward slash "/" is missing.
- We found a user-controlled input instance (source), passed in the code until being parsed inside an `innerHTML` method (sink) insecurely.

To describe the latter's risk, an attacker would follow this scenario:

- Attacker deploys a malicious ERC20 token with symbol name (e.g. `"EVIL<img src=x onerror=alert(document.cookie)>..."`)
- Swapboard is a permissionless protocol and anyone can call `createOrder` to list any ERC20 token without approval
- Victim views the orderbook and the attacker's malicious order
- Victim clicks "Revoke", which triggers the unescaped symbol XSS payload within the vulnerable `showToast` function:

```
function showToast(msg, type = "info", persistent = false) {
  const toast = $("#toast");
  if (msg.endsWith("...")) {
    toast.innerHTML = msg.slice(0, -3) + '<span class="loading-dots"></span>';
  } else {
    toast.textContent = msg;
  }
}
```



An attacker manages to exploit a scenario like portrayed above would be able to execute malicious JavaScript in the user's browser context, allowing him to perform actions in that context like stealing session cookies, sign unauthorized transactions to drain tokens, replace the entire UI with fake wallet prompts to phish seed phrases, etc.

Recommendation:

- Default to **textContent** for all dynamic content unless HTML rendering is explicitly required
- Do a sweep (now and periodically) of the code and ensure that all user input is escaped and sanitized all over the codebase. Especially always escape values before **innerHTML**.
- Ensure full compliance with the specifications of [OWASP XSS prevention cheatsheet](#) on the `escapeHtml` method.
- Implement Content-Security-Policy security header in responses returning from the frontend server to create a browser-enforced security boundary that blocks JavaScript execution from untrusted sources, blocking potential XSS payloads even when input sanitization fails.



AW-M-01: Forced Browsing

Severity: **Medium**

Status: **Fixed**

Retest notes:

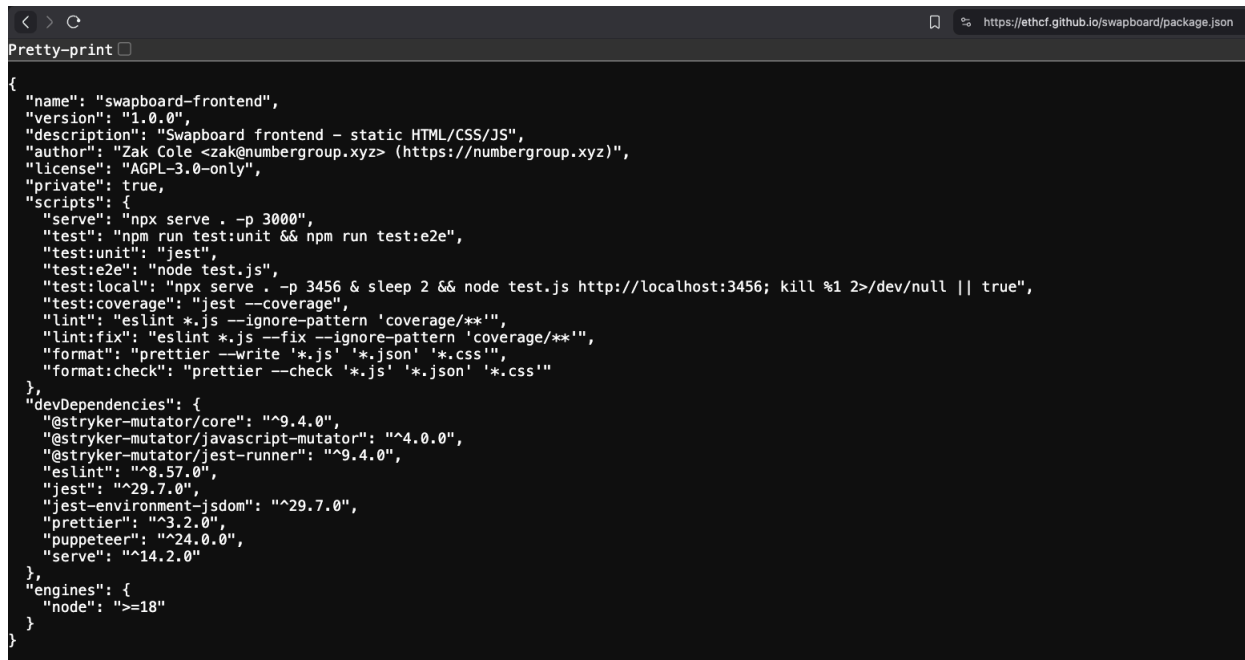
Fixed by implementing whitelist-based deployments that copy only production files to a `dist/` directory, excluding all development files (tests, configs, package.json).

Locations:

- [ETHCF/swapboard/.github/workflows/pages.yml#L6-L8](https://ethcf.swapboard/.github/workflows/pages.yml#L6-L8)
- [ETHCF/swapboard/deploy.sh#L231](https://ethcf.swapboard/deploy.sh#L231)
- <https://ethcf.github.io/swapboard/package.json>
- <https://ethcf.github.io/swapboard/pnpm-lock.yaml>
- <https://ethcf.github.io/swapboard/test.setup.js>

Description:

During the audit it was found that both of Swapboard's deployment mechanisms (GitHub Pages and IPFS) expose development files due to unfiltered directory uploads. The GitHub Actions workflow uses **path: frontend** which uploads the entire frontend directory, while the deployment script uses **ipfs add -r** . which recursively adds all files in the current directory.



```
{
  "name": "swapboard-frontend",
  "version": "1.0.0",
  "description": "Swapboard frontend - static HTML/CSS/JS",
  "author": "Zak Cole <zak@numbergroup.xyz> (https://numbergroup.xyz)",
  "license": "AGPL-3.0-only",
  "private": true,
  "scripts": {
    "serve": "npx serve . -p 3000",
    "test": "npm run test:unit && npm run test:e2e",
    "test:unit": "jest",
    "test:e2e": "node test.js",
    "test:local": "npx serve . -p 3456 & sleep 2 && node test.js http://localhost:3456; kill %1 2>/dev/null || true",
    "test:coverage": "jest --coverage",
    "lint": "eslint *.js --ignore-pattern 'coverage/**'",
    "lint:fix": "eslint *.js --fix --ignore-pattern 'coverage/**'",
    "format": "prettier --write '*.js' '*.json' '*.css'",
    "format:check": "prettier --check '*.js' '*.json' '*.css'"
  },
  "devDependencies": {
    "@stryker-mutator/core": "^9.4.0",
    "@stryker-mutator/javascript-mutator": "^4.0.0",
    "@stryker-mutator/jest-runner": "^9.4.0",
    "eslint": "^8.57.0",
    "jest": "^29.7.0",
    "jest-environment-jsdom": "^29.7.0",
    "prettier": "^3.2.0",
    "puppeteer": "^24.0.0",
    "serve": "^14.2.0"
  },
  "engines": {
    "node": ">=18"
  }
}
```

Example of exposed files found:

- package.json, pnpm-lock.yaml (exact package versions used)
- test.js, lib.unit.test.js, test.setup.js (implementation logic)
- test.config.js, stryker.config.json (configuration fileS)
- __mocks__/ethers.js (test wallet addresses, mock patterns)



- [mutation-report.html](#) (code coverage insights)

Although no current secret was found during the audit as a result of these files being exposed, they open up a point of failure in the future, example risks:

- Developer adds .env files, or writes secrets to package.json, setup scripts, etc.
- Revealing implementation and security testing logic
- Providing attack surface intelligence - when exposing exact package versions such as the ones on package.json and pnpm-lock.yaml, it increases the risk of attackers from the internet to actively find and target the website for CVE exploitation before the protocol owner is even aware of a CVE on its technical stack.

Recommendation:

- Create a proper build script that generates a clean distribution directory. The build should copy only production files (like index.html, app.js, lib.js, style.css, manifest.json, sw.js, API.html, mock.js), exclude all development artifacts automatically, and handle environment-specific configurations.



AW-M-02: Insufficient Contract Security Tests

Severity: **Medium**

Status: **Fixed**

Retest notes:

Tests were re-organized into ``exploit/`` (defensive validation tests) vs ``security-research/ProvenExploits.t.sol`` (known vulnerability documentation) with clear headers and naming conventions. Proven exploits present in a contract still remain dangerous and a risk to the protocol, but as described above they were out of scope of this assessment.

Locations:

- [ETHCF/swapboard/contracts/test/exploit/ExploitVectors.t.sol](#)
- [ETHCF/swapboard/contracts/test/exploit/AdvancedExploits.t.sol](#)

Description:

During the audit it was found that Swapboard's test suite contains systematic flaws that create false security confidence. The testing infrastructure validates that exploitable vulnerabilities work as intended rather than testing that security protections prevent exploitation. The following insufficiencies were found:

- Tests that validate that exploitation is possible (**ExploitVectors.t.sol**):
 - **test_exploit_fot_fillOrder_makerReceivesLess** - Confirms maker receives only 95 tokens instead of expected 100 due to 5% transfer fee. Test passes when maker loses funds rather than verifying protection exists.
 - **test_exploit_pausableToken_blocksOperations** - Validates that paused tokens block both fill and cancel operations. Misses the actual vulnerability: maker funds are permanently locked with no recovery mechanism during pause periods.
- Tests that validate that exploitation is possible (**AdvancedExploits.t.sol**):
 - **test_EXPLOIT_phantomTokenB_freeTokenA** - Demonstrates phantom token returns true without transferring. Taker receives 100 tokenA, maker receives 0 tokenB. The protocol lacks balance verification for tokenB transfers.
 - **test_EXPLOIT_negativeRebase_fundsLocked** - Demonstrates negative rebase causes permanent fund lock. The contract holds 100 tokens that rebase to 50 tokens. Both fillOrder and cancelOrder revert, leaving 50 tokens locked permanently with no recovery mechanism.



- **test_EXPLOIT_upgradeableToken_becomeFOT** - Demonstrates token that changes behavior after order creation. Token operates normally during order creation, then enables 5% transfer fee before fill. Maker receives 95 tokens instead of expected 100 with no protection against post-creation token modifications.
- **test_EXPLOIT_rebaseToken_unfairDistribution** - Demonstrates positive rebase creates unrecoverable locked funds. Two orders for 100 tokens each, after a 20% positive rebase contract holds 240 tokens. Both orders fill at 100 tokens each, leaving 40 tokens permanently stuck in contract with no withdrawal mechanism.

This testing methodology opens multiple points of failure, such as developer false confidence as passing tests create belief that security properties are validated when they are not.

Recommendation:

- Restructure the test suite to validate security properties rather than confirming bugs work.
- To keep documentation of known risks, rename exploit testing logic explicitly to a file like **test/security-research/ProvenExploits.t.sol** with clear documentation that these are known vulnerabilities requiring fixes, not validation tests.



AW-L-01: Missing decrement logic in OrderFilled/OrderCanceled handlers inflates token order counts

Severity: **Low**

Status: **Fixed**

Retest notes:

Fixed by adding ``ordersSelling.minus(ONE)`` and ``ordersBuying.minus(ONE)`` decrements in both ``handleOrderFilled`` and ``handleOrderCanceled`` subgraph handlers.

Locations:

- [ETHCF/swapboard/subgraph/src/mapping.ts#L187-L190](#) (Missing `tokenA.ordersSelling.minus(ONE)` before save)
- [ETHCF/swapboard/subgraph/src/mapping.ts#L195-L199](#) (Missing `tokenB.ordersBuying.minus(ONE)` before save)
- [ETHCF/swapboard/subgraph/src/mapping.ts#L230-L233](#) (Missing entire token counter decrement block)

Description:

During the audit, we discovered missing order selling decrements on the subgraph [mapping.ts](#) functionality, which creates data mismatch when fetching historical order counts through the API (e.g. [API.html](#))

```
subgraph > src > TS mapping.ts > handleOrderFilled
170  export function handleOrderFilled(event: OrderFilled): void {
185      // Update tokenA volume (amount sold)
186      let tokenA = Token.load(order.tokenA);
187  ✓  if (tokenA != null) {
188          tokenA.volumeSold = tokenA.volumeSold.plus(order.amountA);
189  |      // BUG: not decrementing ordersSelling counter
190          tokenA.save();
191  ✓  } else {
192          log.error("OrderFilled: TokenA {} not found for order {}", [order.tokenA, orderId]);
193      }
}
```

The protocol's frontend uses subgraph, which indexes all **OrderCreated**, **OrderFilled**, and **OrderCanceled** events, providing a fast GraphQL API for the frontend to query/display orders.

The data structure defined on Subgraph, maintains ordersSelling and ordersBuying counters that are incremented when orders are created via [handleOrderCreated](#) but are never decremented when orders are filled or cancelled.



An example problematic flow:

- User calls **Swapboard.fillOrder(orderId)**
- The swapboard contract emits **OrderFilled** event
- The public subgraph indexes event, triggering the **handleOrderFilled** call from [mapping.ts](#)
- Order marked as **active: false**
- Token volumes updated (volumeSold, volumeBought)
- Token counter DO NOT decrement as expected

This creates a semantic mismatch where the counters represent cumulative historical totals rather than active order counts.

Although we did not find a mis-representation on the swapboard frontend due to this data (as it appears unused), the protocol does encourage use of its API ([API.html](#)) for market makers and trading bots, which could be affected by the incorrect counter values served through the GraphQL endpoint, leading to incorrect trading decisions. Moreover, if the frontend will ever make use of these values, it might be prone to such inaccuracy as well.

Recommendation:

- To ensure data consistency and accuracy in the subgraph API, implement counter decrements in both **handleOrderFilled** and **handleOrderCanceled** functions on [mapping.ts](#) to mirror the increments performed in **handleOrderCreated**. Ensure to apply this to all locations abovementioned.
- Currently the issue only affects API users, so until fix it's recommended to document the inconsistency on the API docs.





This report was produced by Auditware for ETHCF based solely on the information provided by them. Auditware provides no guarantees as to the accuracy of the contents of this report and does not make any guarantees that following the advice within will prevent security incidents or issues.

Auditware is available for questions or comments about any of the contents of this report. We can be reached at <https://auditware.io/> or by email at joe@auditware.io.

